AtaxxBoard(void) – [TestAtaxxBoard]

Input: N/A

State: N/A

Output: N/A

State:

X						O
		#		#		
		#		#		
O						X

pieceCount{X=2, O=2}

whatAtPos(BoardPosition pos) – [TestwhatsAtPos_0_0_X]

Input: Pos: (0,0)

State:

X						O
		#		#		

Output: X

State:

X						O
		#		#		

		#		#		
O						X

		#		#		
O						X

pieceCount{X=2, O=2}

whatAtPos(BoardPosition pos) – [TestwhatsAtPos_6_6_X]

Input: Pos (6,6)

State:

X						O
		#		#		
		#		#		
O						X

Output: ‘X’

State:

X						O
		#		#		
		#		#		
O						**X**

pieceCount{X=2, O=2}

whatAtPos(BoardPosition pos) – [TestwhatsAtpos_0_6_O]

Input: Pos (0,6)

State:

X						O
		#		#		
		#		#		
O						X

Output: ‘O’

State:

X						O
		#		#		
		#		#		
O						X

pieceCount{X=2, O=2}

whatAtPos(BoardPosition pos) – [TestwhatsAtPos_6_0_O]

Input: Pos(6, 0)

State:

X						O
		#		#		
		#		#		
O						X

Output: ‘O’

State:

X						O
		#		#		
		#		#		
O						X

	pieceCount{X=2, O=2}

whatAtPos(BoardPosition pos) – [TestWhatsAtPos_2_2_Block]

Input: Pos(2,2)

State:

X						O
		#		#		
		#		#		
O						X

Output: '#'

State:

X						O
		#		#		
		#		#		
O						X

pieceCount{X=2, O=2}

placeAtPos(BoardPosition pos, char player) – [TestplaceAtPos_0_0_O]

Input:pos (0,0)

char ‘X’

State:

X						O
		#		#		
		#		#		
O						X

Output:N/A

State:

O						O
		#		#		
		#		#		
O						X

pieceCount{X=2, O=2}(does not change in function)

placeAtPos(BoardPosition pos, char player) – [TestplaceAtPos_6_6_O]

Input:pos (6,6)

char ‘O’

State:

X						O
		#		#		
		#		#		
O						X

Output:N/A

State:

X						O
		#		#		
		#		#		
O						O

	pieceCount{X=2, O=2}

placeAtPos(BoardPosition pos, char player) – [TestplaceAtPos_6_0_X]

Input:pos(6,0)

char ‘X’

State:

X						O
		#		#		
		#		#		
O						X

Output:N/A

State:

X						O
		#		#		
		#		#		
X						X

pieceCount{X=2, O=2}

placeAtPos(BoardPosition pos, char player) – [TestplaceAtPos_0_6_X]

Input:pos(0,6)

char ‘X’

State:

X						O
		#		#		
		#		#		

Output:N/A

State:

X						X
		#		#		
		#		#		

O						X

O						X

pieceCount{X=2, O=2}

placeAtPos(BoardPosition pos, char player) – [TestplaceAtPos_3_3_X]

Input:pos(3,3)

char ‘X’

State:

X						O
		#		#		
		#		#		
O						X

Output:N/A

State:

X						O
		#		#		
			X			
		#		#		
O						X

pieceCount{X=2, O=2}

getPieceCounts(void) – [TestPieceCount]

Input:N/A

State:

X						O
		#		#		
		#		#		

Output:

{X=2, O=2}

State:

X						O
		#		#		
		#		#		

						X
O						X

O						X

isValidMove(BoardPosition pos, char player) – [TestInvalidMove_EmptyNotReachable]

Input: Pos(3,3)

Char: X

State:

X						
		#		#		
		#		#		
O						X

Output: False

State

X						O
		#		#		
		#		#		
O						X

:

A

isValidMove(BoardPosition pos, char player) – [TestValidMove_0_1_X]

Input: Pos(0,1)

Char: X

State:

X	X				O	O
		#		#		
		#		#		
O	O					X

Output:True

State:

X	X				O	O
		#		#		
		#		#		
O	O				X	X

isValidMove(BoardPosition pos, char player) – [TestInvalidMove_OwnPiece]

Input:pos(0,0)

Char:X

State:

X						O
		#		#		
		#		#		
O						X

Output:False

State:

X						O
		#		#		
		#		#		
O						X

isValidMove(BoardPosition pos, char player) – [TestInvalidmove_Block]

<table>
<tr>
<td>Input:POS(2,2)

Char:X

State:
<table>
<tr><td>X</td><td></td><td></td><td></td><td></td><td></td><td>O</td></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td></td><td></td><td>#</td><td></td><td>#</td><td></td><td></td></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td></td><td></td><td>#</td><td></td><td>#</td><td></td><td></td></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td>O</td><td></td><td></td><td></td><td></td><td></td><td>X</td></tr>
</table>
</td>
<td>Output:False

State:
<table>
<tr><td>X</td><td></td><td></td><td></td><td></td><td></td><td>O</td></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td></td><td></td><td>#</td><td></td><td>#</td><td></td><td></td></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td></td><td></td><td>#</td><td></td><td>#</td><td></td><td></td></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td>O</td><td></td><td></td><td></td><td></td><td></td><td>X</td></tr>
</table>
</td>
</tr>
</table>

isValidMove(BoardPosition pos, char player) – [TestValidJumpMove_X]

Input:Pos(2,0)

Char:X

State:

X						O
		#		#		
		#		#		
O						X

Output:True

State:

X						O
		#		#		
		#		#		
O						X

isValidMove(BoardPosition pos, char player) – [TestInvalidOutOfRange]

Input:7

State:

X						O
		#		#		
		#		#		

Output:

State:

X						O
		#		#		
		#		#		

O						X

O						X

convertAdjacentPieces(BoardPosition pos, char player) – [TestConverSinglePiece]

Input:Pos(3,3)
Char:X

State:

X						O
		O				
			X			
		#		#		
O						X

Output:N/A

State:

X						O
		X				
			X			
		#		#		
O						X

convertAdjacentPieces(BoardPosition pos, char player) – [TestCOnvertMultiplePieces]

Input:Pos(3,3)
Char:X

State:

X						O
		O		O		
			X			
		O		O		
O						X

Output:N/A'

State:

X						O
		X		X		
			X			
		X		X		
O						X

convertAdjacentPieces(BoardPosition pos, char player) – [TestNoConversion]

Input:Pos(3,3)

Char:X

State:

X						O
			X			

Output:N/A

State

X						O
			X			

O						X

O						X

:

convertAdjacentPieces(BoardPosition pos, char player) – [TestEdgeConversion]

Input:Pos(0,1)
Char:X

State:

X	O					O
		#		#		
		#		#		
O						X

Output:

N/A

State:

X	X					O
		#		#		
		#		#		
O						X

convertAdjacentPieces(BoardPosition pos, char player) – [TestMixedAdjacentPieces]

Input:Pos(3,3)

State:

X						O
		O		X		
			X			
		O		X		
O						X

Output:N/A

State:

X						O
		X		X		
			X			
		X		X		
O						X

convertAdjacentPieces(BoardPosition pos, char player) – [TestCornerConversion]

Input:Pos(0,1)
Char:X

State:

X	O					O
		#		#		

Output:N/A

State:

O	O					O
		#		#		

		#		#		
O						X

		#		#		
O						X

checkForWin(char player) – [TestNoMorePlayer_O_CheckForWin]

Input:N/A

State

O	O				O	O
O	O				O	O
O	O	#		#	O	O
O						
O		#		#	O	O
O		O				O
O	O	O				O

pieceCount{X=0, O=23}

Output:True

State:

O	O				O	O
O	O				O	O
O	O	#		#	O	O
O						
O		#		#	O	O
O		O				O
O	O	O				O

pieceCount{X=0, O=23}

checkForWin(char player) – [TestPlayerNoMorePlayer_X_CheckForWin]

Input:N/A

State:

X	X				X	X
X	X				X	X
X	X	#		#	X	X
X						
X		#		#	X	X
X		X				X
X	X	X				X

pieceCount{X=23, O=0}

Output:True

State:

X	X				X	X
X	X				X	X
X	X	#		#	X	X
X						
X		#		#	X	X
X		X				X
X	X	X				X

pieceCount{X=23, O=0}

checkForWin(char player) – [TestPlayer_X_most]

Input:N/A

State:

X	X	X	X	X	X	X
X	X	O	O	O	X	X
X	O	#	O	#	O	X
X	X	X	X	X	X	X
X	X	#	X	#	O	X
X	X	X	O	X	X	X
X	X	X	X	X	X	X

pieceCount{X=37, O=12}

Output:True

State:

X	X	X	X	X	X	X
X	X	O	O	O	X	X
X	O	#	O	#	O	X
X	X	X	X	X	X	X
X	X	#	X	#	O	X
X	X	X	O	X	X	X
X	X	X	X	X	X	X

pieceCount{X=37, O=12}

checkForWin(char player) – [TestPlayer_O_most]

Input:N/A

State:

X	X	X	X	X	X	X
O	O	O	O	O	O	O
X	O	#	O	#	O	X
O	O	O	O	O	O	O
X	O	#	O	#	O	X
X	O	O	O	O	O	X
X	X	X	X	X	X	X

pieceCount{X=24, O=25}

Output:True

State:

X	X	X	X	X	X	X
X	O	O	O	O	O	X
X	O	#	O	#	O	X
O	O	O	O	O	O	O
X	O	#	O	#	O	X
X	O	O	O	O	O	X
X	X	X	X	X	X	X

pieceCount{X=24, O=25}

isGameOver(void) – [TestBoardFilled_O]

Input:N/A

State:

O	O	O	O	O	O	O
O	O	O	O	O	O	O
O	O	#	O	#	O	O
O	O	O	O	O	O	O
O	O	#	O	#	O	O
O	O	O	O	O	O	O

Output:True

State:

O	O	O	O	O	O	O
O	O	O	O	O	O	O
O	O	#	O	#	O	O
O	O	O	O	O	O	O
O	O	#	O	#	O	O
O	O	O	O	O	O	O

O	O	O	O	O	O	O

pieceCount{X=0, O=49}

O	O	O	O	O	O	O

pieceCount{X=0, O=49}

isGameOver(void) – [TestBoardfilled_X]

Input:N/A

State:

X	X	X	X	X	X	X
X	X	X	X	X	X	X
X	X	#	X	#	X	X
X	X	X	X	X	X	X
X	X	#	X	#	X	X
X	X	X	X	X	X	X
X	X	X	X	X	X	X

pieceCount{X=49, O=0}

Output:True

State:

X	X	X	X	X	X	X
X	X	X	X	X	X	X
X	X	#	X	#	X	X
X	X	X	X	X	X	X
X	X	#	X	#	X	X
X	X	X	X	X	X	X
X	X	X	X	X	X	X

pieceCount{X=49, O=0}

isGameOver(void) – [TestNoMorePlayer_O_GameOver]

Input:N/A

State:

X	X	X			X	X
X	X	X				X
X	X	#		#		

Output:True

State:

X	X	X			X	X
X	X	X				X
X	X	#		#		

		#		#		
X	X			X	X	
X	X				X	X

pieceCount{X=19, O=0}

		#		#		
X	X			X	X	
X	X				X	X

pieceCount{X=19, O=0}

isGameOver(void) – [TestBoardNoMore_X_GameOver]

Input:N/A

State:

O	O					O
O	O					
		#		#		
		#		#		
O						O
O	O				O	O

pieceCount{X=0, O=11}

Output:True

State:

O	O					O
O	O					
		#		#		
		#		#		
O						O
O	O				O	O

pieceCount{X=0, O=11}

saveBoardToFile(String filename) – [TestSaveFile]

Input:saveBoardToFile”testLoad.txt”

State:

X	X				O	O
		#		#		
		#		#		
O	O				X	X

pieceCount{X=4, O=4}

Output:N/A

State:

X	X				O	O
		#		#		
		#		#		
O	O				X	X

pieceCount{X=4, O=4}

loadBoardFromFile(String filename) – [test loadBoardFromFile]

Input: string “testLoad.txt =

X	X				O	O
		#		#		
		#		#		
O	O				X	X

Output:N/A

State:

X	X				O	O
		#		#		
		#		#		

(the split columns in this string are just spaces no lines)

State:N/A

O	O				X	X

pieceCount{X=4, O=4}

toString(void) – [TesttoString]

Input:N/A

State:

X						O
		#		#		
		#		#		
O						X

Output:

string ”| | 0| 1| 2| 3| 4| 5| 6|

|0 | X| | | | | | O|

|1 | | | | | | | |

|2 | | | | | | | |

|3 | | | | | | | |

|4 | | | | | | | |

|5 | | | | | | | |

|6 | O| | | | | | X|”

State:

X						O
		#		#		
		#		#		

	<table><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>O</td><td></td><td></td><td></td><td></td><td></td><td>X</td></tr></table> pieceCount{X=2, O=2}

What tests did each team member write? Just tell me the names of the functions (unless for some reason multiple team members wrote functions for the same method. In that case, tell me which tests specifically by giving me the test names)

Richard Gibbs	placeAtPos toString loadBoardFromFile
William chen	checkForWin isGameOver saveBoardToFile(String filename)
Simon Lu	AtaxxBoard whatsAtPos

[Ayan Ngariya]	isValidMove convertAdjacentPieces